

Crocette di Sicurezza Informatica

Raccolta di domande V/F da esami precedenti

Cruciat Elia

29 maggio 2026

Indice

1	Autenticazione e Controllo degli Accessi	3
1.1	Password e Salt	3
1.2	Challenge & Response	3
1.3	Comandi Unix e Password	3
1.4	Fattori di Autenticazione	3
1.5	FIDO Alliance e OTP	4
1.6	Autenticazione Attiva	4
1.7	2FA vs 2SA	4
1.8	Controllo degli Accessi: Paradigmi	4
1.9	ACL e Capability List	5
1.10	RBAC	5
1.11	Permessi Unix e NTFS	5
1.12	Bit SUID e SGID	5
2	Crittografia	6
2.1	Cifrari a Sostituzione e Frequenze	6
2.2	Cifrari a Trasposizione	6
2.3	Cifrari Polialfabetici	6
2.4	Test di Kasiski e Indice di Coincidenza	6
2.5	Diffusione e Confusione	7
2.6	Cifrari a Blocchi	7
2.7	RSA e Diffie-Hellman	7
2.8	Crittografia e Proprietà di Sicurezza	8
2.9	PKI, Certificati e Web of Trust	8
3	Sicurezza di Rete e Attacchi	9
3.1	Attacchi Passivi e Sniffing	9
3.2	IP Spoofing e ARP Poisoning	9
3.3	Firewall	9
3.4	IDS, IPS e SIEM	10
3.5	Botnet e Command & Control	10
3.6	HTTPS e TLS	10
3.7	DNS Exfiltration e SQL Injection	11
4	Sicurezza del Sistema Operativo e Exploit	12
4.1	Secure Boot e Trusted Boot	12
4.2	Attacchi Fisici e USB	12
4.3	Cifratura dei Dischi	12
4.4	Cloud e Disponibilità	12
4.5	Protezioni Memoria: Canarini e ASLR	13
4.6	Buffer Overflow e NOP Sled	13
4.7	Brute Force e Protezioni	13
4.8	Kernel e System Call	14
4.9	HIDS e Integrità dei File	14

4.10 Log e Monitoraggio	14
4.11 Vulnerabilità Web: File Inclusion e Code Injection	14

Capitolo 1 - Autenticazione e Controllo degli Accessi

1.1 Password e Salt

1) Il salt è una variazione random inserita dal sistema all'atto della memorizzazione della password scelta. → **VERO** (freq. 3)

66) Il salt è una password aggiuntiva di secondo livello. → **FALSO**

Il salt è una variazione *random* concatenata alla password prima dell'hashing, **non** un secondo segreto scelto dall'utente.

1.2 Challenge & Response

2) I sistemi a sfida e risposta sono tipicamente implementati condividendo un segreto come ad esempio una chiave simmetrica. → **FALSO** (freq. 3)

I sistemi a sfida e risposta utilizzano tipicamente la crittografia **asimmetrica**: la challenge è cifrata con la chiave pubblica del Prover, che la decifra con la propria chiave privata. La chiave simmetrica viene condivisa con altri meccanismi (es. Diffie-Hellman).

1.3 Comandi Unix e Password

3) In Unix il comando `passwd` si usa per verificare la robustezza della password. → **FALSO** (freq. 2)

Il comando `passwd` serve a **cambiare** la password di un utente, non a valutarne la robustezza.

1.4 Fattori di Autenticazione

4) Tra i fattori di autenticazione c'è qualcosa che si conosce (Password, PIN). → **VERO**

73) Tra i fattori di autenticazione c'è qualcosa che si possiede fisicamente, come un Pin o una Password. → **FALSO** (freq. 2)

Un pin o una password sono classificati come **qualcosa che si sa** (*knowledge factor*), non come qualcosa che si possiede fisicamente. Il possesso fisico riguarda token hardware, smart card, YubiKey, ecc.

1.5 FIDO Alliance e OTP

5) FIDO Alliance è un sistema di generazione degli OTP. → **FALSO** (freq. 2)

La **FIDO Alliance** è un consorzio di aziende (Google, Microsoft, ecc.) che sviluppa *standard aperti* per un'autenticazione più semplice e sicura, come U2F e UAF. Non ha nulla a che vedere con la generazione di OTP.

42) Lo scopo del TOTP è quello di forzare l'utente a cambiare periodicamente la password. → **FALSO**

Con **TOTP** si intende la *Time-based One-Time Password*: un token aggiuntivo valido per un solo utilizzo e con un breve limite temporale. Non ha alcuna relazione con il cambio periodico della password.

1.6 Autenticazione Attiva

41) Nell'autenticazione attiva Prover e Verifier si scambiano ogni volta un dato diverso. → **VERO** (freq. 2)

Nell'autenticazione attiva il Prover dimostra il possesso del segreto senza mai trasmetterlo direttamente, inviando ogni sessione un valore diverso. Questo rende inutili gli attacchi di replay.

1.7 2FA vs 2SA

74) 2FA e 2 step authentication sono esattamente la stessa cosa. → **FALSO** (freq. 2)

Nel **2-Step Authentication** viene aggiunto un livello ulteriore ma non necessariamente di categoria diversa. Nel vero **2FA** i due fattori devono essere categorialmente distinti (es. qualcosa che si sa + qualcosa che si ha fisicamente).

1.8 Controllo degli Accessi: Paradigmi

6) Il controllo dell'accesso è decidere se un soggetto può eseguire una specifica operazione su di un oggetto. → **VERO**

9) I due paradigmi fondamentali per il controllo dell'accesso sono DAC (Discretionary Access Control) e TAC (Tertiary Access Control). → **FALSO**

I due paradigmi fondamentali sono:

- **DAC (Discretionary Access Control)**: ogni oggetto ha un proprietario che decide i permessi.
 - **MAC (Mandatory Access Control)**: la proprietà di un oggetto non consente di modificarne i permessi; esiste una policy centralizzata gestita da un security manager.
-

1.9 ACL e Capability List

10) Le capability list sono delle liste associate a ogni soggetto del sistema. → **VERO** (freq. 2)

43) Le ACL sono liste associate ad ogni soggetto del sistema. → **FALSO** (freq. 3)

44) Le ACL e le Capabilities sono la stessa cosa. → **FALSO** (freq. 2)

Le **ACL** (Access Control List) sono associate a ogni *risorsa/oggetto*, mentre le **Capability List** sono associate a ogni *soggetto*. Sono strumenti complementari e diversi.

1.10 RBAC

76) Nel modello RBAC (Role-based Access Control) i permessi sono assegnati ai ruoli. → **VERO**

1.11 Permessi Unix e NTFS

45) I bit di autorizzazione standard nel filesystem Unix sono di 3 tipi R, W, X (read, write, execute). → **VERO** (freq. 2)

46) Ogni permesso su di un file in NTFS può assumere due soli valori (concesso o negato). → **FALSO** (freq. 2)

In NTFS ogni permesso può assumere tre valori: **allow**, **deny** o **non impostato**. La distinzione tra deny esplicito e non impostato è fondamentale per la risoluzione dei conflitti.

1.12 Bit SUID e SGID

7) Il comando `find / -type f -perm /6000` permette di trovare i file coi bit SUID e SGID impostati a 1. → **FALSO**

Nota: questa domanda è controversa. Il comando nella sua forma attuale trova effettivamente i file con SUID o SGID impostati. La risposta corretta varia a seconda della versione dell'esame.

8) Il SUID è il bit che permette di lanciare con `sudo` il programma su cui è settato. → **FALSO** (freq. 3)

Il **SUID** fa eseguire il programma con i permessi dell'**owner del file**, non con `sudo`. Esempio classico: `/usr/bin/passwd` ha SUID impostato ed è di proprietà di root.

Capitolo 2 - Crittografia

2.1 Cifrari a Sostituzione e Frequenze

13) Nell'attacco dei cifrari a sostituzione il fatto che alcune lettere siano più frequenti di altri nel linguaggio naturale non ha nessuna importanza. → **FALSO**

La **frequenza delle lettere** è lo strumento principale per attaccare i cifrari monoalfabetici: le lettere più frequenti nel cifrato corrispondono alle lettere più frequenti nella lingua naturale.

14) Nel cifrario con sostituzione monoalfabetica sull'alfabeto inglese lo spazio delle chiavi è grande $26!$. → **VERO** (*freq. 2*)

Con 26 lettere, il numero di permutazioni possibili è $26! \approx 4 \times 10^{26}$, uno spazio enormemente grande ma comunque vulnerabile all'analisi delle frequenze.

2.2 Cifrari a Trasposizione

11) Nei cifrari a trasposizione le statistiche dei digrammi e trigrammi permettono di dedurre la dimensione della tabella di cifratura. → **VERO** (*freq. 3*)

2.3 Cifrari Polialfabetici

47) Nei cifrari a sostituzione polialfabetica conoscere il contenuto di una parte del messaggio non aiuta la decifrazione dell'intero testo. → **FALSO** (*freq. 3*)

Anche nei cifrari polialfabetici la conoscenza parziale del testo aiuta, perché permette di individuare la **ripetizione periodica** delle sostituzioni e quindi di risalire alla lunghezza della chiave.

49) Nei cifrari a sostituzione polialfabetica le frequenze di un carattere cifrato derivano da contributi di diversi caratteri in chiaro. → **VERO**

Questo è esattamente il punto di forza dei cifrari polialfabetici rispetto ai monoalfabetici: ogni carattere del plaintext può produrre caratteri cifrati diversi a seconda della posizione.

2.4 Test di Kasiski e Indice di Coincidenza

15) Nel test di Kasiski si impiega la fattorizzazione delle distanze e la scelta di quelle con un fattore comune. → **VERO** (*freq. 3*)

Il **test di Kasiski** individua sequenze ripetute nel cifrato, calcola le distanze tra le loro occorrenze e fattorizza tali distanze per dedurre la lunghezza della chiave.

48) L'Indice di Coincidenza è la probabilità che due lettere scelte a caso in un testo siano *diverse*. → **FALSO** (freq. 2)

Trabocchetto: l'Indice di Coincidenza è la probabilità che due lettere scelte a caso in un testo siano **uguali**.

2.5 Diffusione e Confusione

12) La proprietà di diffusione misura il grado in cui le proprietà statistiche degli elementi del testo cifrato vengono sparse sugli elementi del testo in chiaro. → **FALSO**

La direzione è invertita: la **diffusione** misura come le proprietà statistiche del *testo in chiaro* vengono disperse sugli elementi del *testo cifrato*. Il testo cifrato non può influenzare in retroazione il testo in chiaro.

2.6 Cifrari a Blocchi

78) Gli algoritmi di cifratura a blocchi sono così definiti perché si arrestano non appena la cifrazione è considerata sicura. → **FALSO**

I cifrari a blocchi sono così chiamati perché cifrano il testo dividendolo in **blocchi di dimensione fissa**. Non esiste alcun criterio di “sicurezza raggiunta” che li faccia fermare.

20) CBC sta per Cipher Block Chaining e consiste nel cifrare un blocco modificandolo col contributo del blocco cifrato precedente. → **VERO** (freq. 2)

In modalità **CBC**, prima di cifrare ogni blocco di plaintext viene effettuato uno XOR con il blocco cifrato precedente (o con l'IV per il primo blocco), creando una catena di dipendenze.

2.7 RSA e Diffie-Hellman

16) Il miglior attacco a RSA è la ricerca dei fattori del modulo. → **VERO** (freq. 3)

La sicurezza di RSA si basa sulla difficoltà computazionale della **fattorizzazione** di grandi numeri. Fattorizzare $n = p \cdot q$ permetterebbe di derivare la chiave privata.

53) Uno dei presupposti per la robustezza degli algoritmi a chiave asimmetrica è che non esistono modi efficienti di fattorizzare il modulo. → **VERO**

17) DH e RSA hanno scopi differenti: quello di DH è scambiare una chiave condivisa tra due parti. → **VERO**

18) Diffie-Hellman è uno schema di cifratura a chiave simmetrica. → **FALSO**

DH è un protocollo di **scambio di chiave** basato su crittografia asimmetrica: permette a due parti di concordare un segreto condiviso su un canale non sicuro, senza cifrare direttamente i messaggi.

19) DH e RSA hanno scopi differenti: quello di RSA è di essere molto più veloce nella fase di cifratura/decifrazione. → **FALSO** (freq. 3)

79) DH e RSA hanno scopi differenti: quello di RSA è di scambiarsi la chiave simmetrica di cifratura. → **FALSO**

RSA e DH hanno scopi distinti ma non sono direttamente comparabili: RSA è usato per firma e cifratura asimmetrica, DH per lo scambio di chiavi. RSA è inoltre molto più **lento** della crittografia simmetrica.

65) Data chiave pubblica (e, n) e chiave privata (d, n) la cifratura consiste nel calcolare: $c = m^e \bmod n$. → **VERO**

2.8 Crittografia e Proprietà di Sicurezza

50) La crittografia permette di proteggere le quattro proprietà di sicurezza dell'informazione: riservatezza, integrità, autenticità e disponibilità. → **FALSO** (freq. 2)

La crittografia **non protegge la disponibilità** delle informazioni. La disponibilità (availability) è garantita da meccanismi di ridondanza, backup, ecc.

77) I cifrari classici possono essere impiegati per proteggere riservatezza e autenticità. → **FALSO**

I cifrari classici non proteggono l'**autenticità**: non fanno uso di chiave asimmetrica e non possono fornire una firma digitale verificabile.

2.9 PKI, Certificati e Web of Trust

51) Nel modello Infrastrutturale della certificazione delle chiavi pubbliche l'autenticità della chiave pubblica è data da un soggetto terzo fidato che emette la certificazione. → **VERO** (freq. 2)

Il soggetto terzo fidato è la **Certification Authority (CA)**. La fiducia nella CA radice è implicita e basata su accordi istituzionali.

52) Nel modello web of trust della certificazione delle chiavi pubbliche l'autenticità della chiave pubblica è attestata dagli altri utenti. → **VERO** (freq. 2)

Il modello **Web of Trust** (tipicamente usato con PGP/GPG) si basa sulla fiducia transitiva tra utenti, senza una CA centralizzata.

58) X.509 è la versione di SSL più utilizzata. → **FALSO**

X.509 non è una versione di SSL: è uno standard che definisce il formato dei certificati a chiave pubblica ed è alla base dell'infrastruttura PKI.

67) La fiducia nell'autenticità di una Root Certification Authority è perfettamente verificabile dal corrispondente certificato. → **FALSO**

Una Root CA **firma sé stessa**: non è possibile verificarne l'autenticità tramite il certificato. Ci si fida ciecamente delle Root CA preinstallate nel sistema operativo o nel browser.

Capitolo 3 - Sicurezza di Rete e Attacchi

3.1 Attacchi Passivi e Sniffing

33) Lo sniffing può compromettere la riservatezza dei dati. → **VERO** (freq. 2)

34) Gli attacchi passivi non modificano i dati in transito. → **VERO** (freq. 4)

Gli attacchi passivi (come lo sniffing) si limitano a **osservare** il traffico senza alterarlo, violando la *riservatezza* ma non l'integrità.

35) Lo sniffing NON richiede accesso fisico alla rete. → **FALSO** (freq. 2)

Trabocchetto: per sniffare è necessario essere **connessi alla rete** su cui viaggiano i pacchetti. Se la comunicazione è wireless, occorre essere nelle vicinanze fisiche dello scambio.

3.2 IP Spoofing e ARP Poisoning

32) L'IP spoofing consiste nel cercare di scoprire l'IP di una macchina vittima. → **FALSO**

69) L'IP spoofing consiste nell'assumere un indirizzo IP diverso da quello regolarmente assegnato al proprio sistema. → **VERO**

83) Appropriarsi dell'indirizzo di un host può permettere di attaccarlo di riflesso. → **VERO**

75) L'ARP poisoning consiste nel convincere un host che l'IP di una vittima è associato al MAC dell'attaccante. → **VERO**

L'ARP poisoning sfrutta la mancanza di autenticazione nel protocollo ARP: inviando risposte ARP fasulle, l'attaccante associa il proprio MAC address all'IP della vittima, reindirizzando il traffico verso di sé.

3.3 Firewall

25) L'approccio default deny su firewall significa che tutto il traffico viene bloccato. → **FALSO** (freq. 2)

Trabocchetto: il **default deny** significa che se nessuna regola corrisponde al pacchetto, questo viene bloccato per default. Il traffico che corrisponde a una regola permissiva viene invece inoltrato.

80) Esistono tre tipi fondamentali di firewall: Packet filter, Application-level gateway, Circuit-level gateway. → **VERO**

3.4 IDS, IPS e SIEM

26) Un Intrusion Detection System può bloccare un attacco in corso. → **FALSO** (freq. 2)

Un **IDS** rileva e segnala l'attacco ma non interviene. È invece un **IPS** (Intrusion Prevention System) ad avere la capacità di bloccare attivamente il traffico malevolo.

27) Un vantaggio degli Host-based IDS è che possono classificare più accuratamente il rischio associato a un pacchetto di rete. → **VERO** (freq. 3)

Un HIDS opera direttamente sull'host e ha quindi pieno contesto sull'effetto finale di un pacchetto, riducendo i falsi positivi rispetto a un NIDS.

28) Il tasso di "falsi positivi" non è un parametro importante per la qualità di un IDS. → **FALSO** (freq. 3)

Il tasso di falsi positivi è **fondamentale**: troppi alert su eventi innocui causano *alert fatigue* e portano gli operatori a ignorare il sistema.

29) Un vantaggio dei Network-based IDS è che non interferiscono col funzionamento dei sistemi monitorati. → **VERO**

30) Suricata può funzionare sia da IDS che da IPS. → **VERO** (freq. 2)

55) I sistemi SIEM raccolgono dati che devono essere nativamente omogenei per poter essere confrontati e aggregati. → **FALSO** (freq. 2)

Il **SIEM** raccoglie dati eterogenei da fonti multiple (log, eventi, flussi di rete) e li *normalizza* internamente per rilevare correlazioni.

81) Un EDR può essere definito come una variante evoluta di Network-based IDS. → **FALSO**

Un **EDR** (Endpoint Detection and Response) è una variante evoluta di **HIDS**, non di NIDS: opera a livello di host con integrazione profonda nel kernel.

3.5 Botnet e Command & Control

60) Le botnet sono reti di computer infetti chiamati zombie. → **VERO** (freq. 2)

82) Il Command and Control è il computer incaricato di gestire le comunicazioni con gli elementi di una botnet. → **VERO**

3.6 HTTPS e TLS

31) HTTPS è HTTP su di un canale di comunicazione cifrato. → **VERO**

85) Se un sito è protetto da TLS non è possibile eseguire una SQL Injection. → **FALSO**

TLS protegge il **canale di comunicazione** (riservatezza e integrità in transito), ma non ha alcun effetto sulla vulnerabilità applicativa della SQL Injection, che riguarda il lato server.

802.1x: 59) 802.1x è uno standard di autenticazione utilizzato nelle connessioni fisiche.
→ **VERO**

3.7 DNS Exfiltration e SQL Injection

61) È possibile esfiltrare dati attraverso richieste DNS. → **VERO**

La **DNS exfiltration** sfrutta il fatto che le query DNS sono spesso permesse anche da firewall restrittivi: i dati vengono codificati nei sottodomini delle query.

62) Nelle time based SQL Injection è importante il tempo di computazione della query SQL. → **VERO**

Nelle **time-based blind SQLi** l'attaccante inferisce informazioni dai tempi di risposta: se la query impiega più tempo (es. a causa di un **SLEEP()**), la condizione iniettata è vera.

71) Nelle SQL Injection di tipo union select, il numero di colonne da usare per la query è un dato fondamentale per la riuscita dell'attacco. → **VERO** (*freq. 2*)

L'operatore **UNION** richiede che le due **SELECT** abbiano lo stesso numero e tipo di colonne: determinare questo numero è il primo passo dell'attacco union-based.

Capitolo 4 - Sicurezza del Sistema Operativo e Exploit

4.1 Secure Boot e Trusted Boot

21) Le chiavi di autenticazione usate da Secure Boot sono aggiornabili senza interruzioni di servizio. → **FALSO** (freq. 4)

Per aggiornare le chiavi di Secure Boot, l'immagine del firmware UEFI deve essere firmata con le chiavi private del fornitore e la chiave pubblica va iscritta nel firmware. Questo processo **richiede un reboot**.

54) Secure Boot è il nome specifico dato all'implementazione di trusted boot basata su UEFI. → **VERO**

68) Measured Boot si riferisce a un processo generale, che tipicamente usa un TPM come hardware root of trust. → **VERO**

4.2 Attacchi Fisici e USB

22) È possibile danneggiare fisicamente un sistema attraverso una porta USB. → **VERO** (freq. 2)

Attacchi come **USB Killer** inviano picchi di tensione attraverso la porta USB, causando danni hardware permanenti.

4.3 Cifratura dei Dischi

24) La cifratura dei dischi protegge da qualsiasi tentativo di esfiltrazione dei dati. → **FALSO** (freq. 4)

La cifratura dei dischi protegge il dato *a riposo*, ma non è un rimedio assoluto: un attaccante con accesso fisico al disco può tentare attacchi di **brute force** sulla chiave, con tutto il tempo necessario.

4.4 Cloud e Disponibilità

23) La collocazione di sistemi in cloud migliora (generalmente) la disponibilità dei servizi. → **VERO** (freq. 3)

70) La collocazione di sistemi in cloud ha unicamente effetti positivi sulla sicurezza. → **FALSO**

Il cloud introduce rischi specifici come il **vendor lock-in**, la perdita di controllo diretto sull'infrastruttura, rischi giurisdizionali e l'Economic Denial of Service.

4.5 Protezioni Memoria: Canarini e ASLR

36) I canarini sono un meccanismo di protezione del kernel Linux per segnalare un overflow in memoria. → **FALSO** (*freq. 4*)

I **canarini** (stack canaries) vengono inseriti dal **compilatore** (es. GCC con l'opzione `-fstack-protector`), non dal kernel. Sono valori sentinella posti tra il buffer e il return address.

37) ASLR è un meccanismo di protezione del kernel Linux per randomizzare gli spazi di memoria. → **VERO** (*freq. 2*)

L'**ASLR** (Address Space Layout Randomization) è implementato a livello di kernel e randomizza gli indirizzi base di stack, heap e librerie ad ogni esecuzione, rendendo difficile prevedere gli indirizzi target degli exploit.

4.6 Buffer Overflow e NOP Sled

38) Il comando con codice esadecimale 0x90 dell'assembler x86 è del tutto inutile ai fini della realizzazione di payload. → **FALSO**

0x90 è l'istruzione **NOP** (No Operation) di x86. Il **NOP sled** è una tecnica fondamentale nei buffer overflow: si inserisce una sequenza di NOP prima del shellcode per aumentare la probabilità che l'EIP sovrascritto punti in una zona valida del payload.

40) La `strcpy` in linguaggio C non è una funzione a rischio di generare vulnerabilità di buffer overflow. → **FALSO** (*freq. 3*)

`strcpy` è una delle funzioni C più pericolose: non verifica che la stringa sorgente non superi la dimensione del buffer di destinazione. Si consiglia di usare `strncpy` o funzioni analoghe con controllo della lunghezza.

63) Il registro EIP in x86 salva il valore dell'indirizzo di ritorno da una funzione. → **FALSO**

EIP (Extended Instruction Pointer) punta alla *prossima istruzione da eseguire*, non all'indirizzo di ritorno. L'indirizzo di ritorno viene salvato sullo stack al momento della chiamata di funzione.

4.7 Brute Force e Protezioni

39) Inserire un ritardo di pochi secondi tra due login errate non è una misura efficace per mitigare gli attacchi brute force. → **FALSO**

Un ritardo di anche soli 2-3 secondi riduce drasticamente la velocità di un attacco brute force automatizzato. Questo meccanismo (*rate limiting*) è ampiamente usato in produzione.

4.8 Kernel e System Call

56) Il kernel di Linux permette di porre sotto monitoraggio l'invocazione di ogni system call. → **VERO** (*freq. 2*)

Questo è reso possibile da **auditd**, il sottosistema di auditing del kernel Linux che permette di registrare e monitorare le system call invocate dai processi.

4.9 HIDS e Integrità dei File

57) Il controllo dell'integrità dei file è uno dei metodi usati dagli HIDS. → **VERO** (*freq. 2*)

Strumenti come **AIDE** o **Tripwire** calcolano hash dei file critici e rilevano modifiche non autorizzate.

4.10 Log e Monitoraggio

72) I log sono utili solo a fini forensi (cioè per comprendere un attacco dopo che si è compiuto). → **FALSO**

I log possono essere analizzati **in tempo reale** da un HIDS o un SIEM per rilevare attacchi in corso, non solo a posteriori.

4.11 Vulnerabilità Web: File Inclusion e Code Injection

64) Con una Local File Inclusion possiamo recuperare file interni al web server. → **VERO** (*freq. 2*)

Una **LFI** sfrutta parametri non sanificati per risalire l'albero delle directory (../) e accedere a file interni come `/etc/passwd`.

84) Code Injection si verifica quando dell'input non sanitizzato viene interpretato come codice. → **VERO**

Il problema generale dell'**injection** (SQL, Command, Code) è la mancata separazione tra dati e istruzioni: l'input dell'utente viene interpretato come parte del programma.